

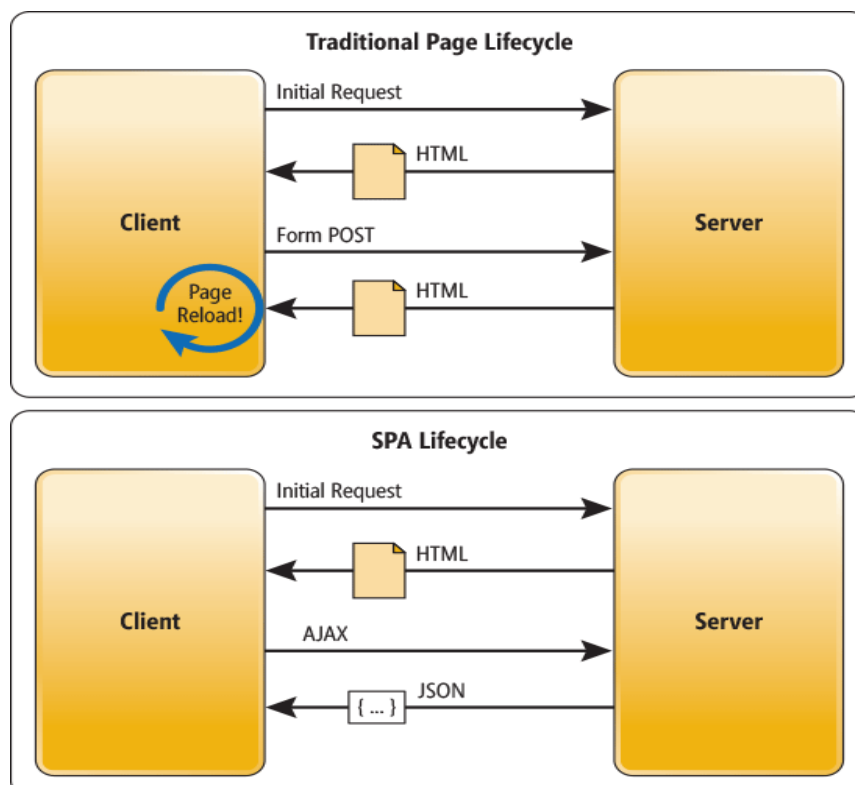
Unit-5: Angular JS: Single page application:

5.1 Single page application using AngularJS

- Traditionally, applications were Multi-Page Applications (MPA) where with every click a new page would be loaded from the server. This was not only time-consuming but also increased the server load and made the website slower.
- AngularJS is a powerful javascript framework for building dynamic web applications. It became insanely popular nowadays. The good thing about Angular is that it has a set of ready-to-use modules to simplify building of single page applications.

What is SPA (Single page application) in AngularJS?

- Single page application (SPA) is a web application that fits on a single page.
- AngularJS is a powerful javascript framework, used for developing Single Page Application (SPA) projects in very clean and maintainable way.
- Single Page Application is a web application or website that interacts with the user by dynamically rewriting the current page rather than loading entire new pages from a server).
- All your code (JS, HTML, CSS) is retrieved with a single page load. And navigation between pages performed without refreshing the whole page.
- The page does not reload or transfer control to another page during the process. This ensures high performance and loading pages faster. Most modern applications use the concept of SPA.



When to use SPA?

- SPAs are good when the volume of data is small and the website needs a dynamic platform.
- It is also a good option for mobile applications. But businesses that depend largely on search engine optimizations such as e-commerce applications must avoid single-page applications and opt for MPAs.

Popular Single Page Application

- Facebook
- Gmail
- Twitter
- Google Drive
- Netflix

Single Page Application Architecture and How SPA works

The architecture of AngularJS is designed to create dynamic, single-page applications (SPAs) with a focus on separation of concerns and efficient data binding.

Here's an overview of the key components and concepts that make up AngularJS architecture:

1. MVC Pattern

AngularJS is built around the Model-View-Controller (MVC) pattern:

- **Model:** Represents the data and business logic of the application. In AngularJS, this typically includes JavaScript objects and services that handle data manipulation and storage.
- **View:** Represents the user interface. In AngularJS, this is often defined using HTML templates combined with AngularJS directives that dynamically render data and handle user interactions.
- **Controller:** Manages the data and behavior for a particular view. Controllers in AngularJS are responsible for initializing the model and handling user input, interacting with services, and updating the view accordingly.

2. Modules

- Modules in AngularJS are containers for different parts of an application, such as controllers, services, filters, and directives.
- Each AngularJS application typically has at least one module, the main module, but can include multiple modules to better organize and manage the application code.

```
var app = angular.module('myApp', []);
```

3. Directives

- Directives are special tokens in the markup that tell the AngularJS compiler to do something to the DOM. They extend HTML with new attributes and elements, and can be used to create reusable components. Examples include:
 - ng-model for two-way data binding.
 - ng-repeat for iterating over arrays.
 - ng-show and ng-hide for conditional display.

```
<div ng-controller="MyCtrl">
    <p ng-bind="message"></p>
</div>
```

4. Controllers

- Controllers in AngularJS are JavaScript functions that initialize the scope and manage the interactions between the view and the model.
- They are defined using the .controller method of an AngularJS module.

```
app.controller('MyCtrl', function($scope) {
    $scope.message = 'Hello, World!';
});
```

5. Services

Services are singleton objects or functions that are used to share data and logic across different parts of an application. Services are typically used for making HTTP requests, managing application state, and performing business logic.

```
app.service('MyService', function($http) {
    this.getData = function() {
        return $http.get('/api/data');
    };
});
```

6. Filters

Filters are used to format data displayed to the user. They can be applied within expressions and directives to transform data in the view. Common filters include:

- currency
- date
- filter

```
<p>{{ amount | currency }}</p>
```

7. Routing

AngularJS provides routing capabilities through the ngRoute module. This module enables you to map URLs to views and controllers, allowing for a seamless navigation experience in SPAs.

```
app.config(function($routeProvider) {  
  $routeProvider  
    .when('/home', {  
      templateUrl: 'home.html',  
      controller: 'HomeCtrl'  
    })  
    .when('/about', {  
      templateUrl: 'about.html',  
      controller: 'AboutCtrl'  
    });  
});
```

8. Dependency Injection

AngularJS uses dependency injection (DI) to manage the components and services within an application. DI allows you to inject dependencies into controllers, services, and other components, making it easier to manage and test the application.

```
app.controller('MyCtrl', function($scope, MyService) {  
  $scope.data = MyService.getData();  
});
```

Single-Page Application Vs Multi-Page Application

- Although SPAs are prevalent these days, it does not mean it is the perfect web development choice for your next project.
- Choosing between a single-page application vs a multi-page application depends on your business requirements.

What is a Multi-Page Application?

A multi-page Application is a traditional web application where a new page is requested from the server to display when data is exchanged back and forth. The amount of content they carry is enormous. Thus, they are generally multi-level deep with many links and intricate UIs.

Advantages of Single Page Application:

1. **Team collaboration:** Single-page applications are excellent when more than one developer is working on the same project. It allows backend developers to focus on the API, while frontend developers can focus on creating the user interface based on the backend API.
2. **Caching:** The application sends a single request to the server and stores all the received information in the cache. This proves beneficial when the client has poor network connectivity.
3. **Fast and responsive:** As only parts of the pages are loaded dynamically, it improves the website's speed.
4. **Debugging is easier:** Debugging single-page applications with chrome is easier since such applications are developed using AngularJS Batarang and React developer tools.
5. **Linear user experience:** Browsing or navigating through the website is easy.

Disadvantages of Single Page Application:

1. **SEO optimization:** SPAs provide poor SEO optimization. This is because single-page applications operate on JavaScript and load data at once server. The URL does not change and different pages do not have a unique URL. Hence it is hard for the search engines to index the SPA website as opposed to traditional server-rendered pages.
2. **Browser history:** A SPA does not save the users' transition of states within the website. A browser saves the previous pages only, not the state transition. Thus when users click the back button, they are not redirected to the previous state of the website. To solve this problem, developers can equip their SPA frameworks with the HTML5 History API.
3. **Security issues:** Single-page apps are less immune to cross-site scripting (XSS) and since no new pages are loaded, hackers can easily gain access to the website and inject new scripts on the client-side.
4. **Memory Consumption:** Since the SPA can run for a long time sometimes hours at a time, one needs to make sure the application does not consume more memory than it needs. Else, users with low-memory devices may face serious performance issues.
5. **Disabled Javascript:** Developers need to chalk out ideas for users to access the information on the website for browsers that have Java script disabled.

5.1.1 Create a module, Define Simple controller

To create an AngularJS module and define a simple controller

1. Create an AngularJS Module: An AngularJS module is a container for different parts of your application, such as controllers, services, and directives. To create a module, use the **'angular.module'** function.

2. Define a Simple Controller: A controller in AngularJS is a JavaScript function that is used to manage the data and behavior for a particular view. It is defined using the **'controller'** method of an AngularJS module.

1. Create the HTML File (index.html)

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>AngularJS Example</title>
  <script src="angular.min.js"></script>
  <script src="app.js"></script>
</head>
<body>
  <div ng-controller="SimpleController">
    <h1>{{ greeting }}</h1>
    <p>{{ message }}</p>
  </div>
</body>
</html>
```

2. Create the AngularJS Application (app.js)

```
// Define the AngularJS module
var app = angular.module('myApp', []);

// Define a simple controller
app.controller('SimpleController', function($scope) {
  // Initialize data and functions on the scope
  $scope.greeting = 'Hello, World!';
  $scope.message = 'Welcome to the AngularJS Application.';
});
```

5.1.2 Embedding AngularJS script in HTML

Embedding AngularJS directly into an HTML file is straightforward. You simply need to include AngularJS via a script tag and then define your AngularJS module and controllers either within the HTML file or in an external JavaScript file.

The ng-bind-html Directive is used to bind the innerHTML of an HTML element to application data and remove dangerous code from the HTML string. The \$sanitize service is a must for the ng-bind-html directive. It is supported by all HTML elements.

Syntax:

<element ng-bind-html="expression"> Contents... </element>

Approach:

- Initialize the libraries that we are using. Here, we are mainly using the angular-sanitize.js library.
- Create an app and its controller scope.
- Define the controller variable.
- Call the app and controller to the body of HTML.
- Inside the body use span tag and use attribute ng-bind-html and assign the value as the scope variable.

Basic Example: Embedding AngularJS in HTML

1. HTML File: (index.html)

```
<html ng-app="myApp">
<head>
  <script src="angular.min.js"></script>
  <script>    // Define the AngularJS module and controller within the HTML file
    angular.module('myApp', [])
      .controller('SimpleController', function($scope) {
        $scope.greeting = 'Hello, AngularJS!';
        $scope.message = 'This is a simple AngularJS example embedded in HTML.';
      });
</script></head>
<body>
  <div ng-controller="SimpleController">
    <h1>{{ greeting }}</h1>
    <p>{{ message }}</p>
```

// Using ng-bind:

```
<h1 ng-bind="greeting"></h1>
<p ng-bind="message"></p>
```

// ng-bind: Automatically replaces the content of the element with the expression's value once it is available, avoiding the initial flickering.

```
</div>
</body>
</html>
```

Using External JavaScript File**1. HTML File (index.html)**

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>AngularJS Example</title>
  <script src="angular.min.js"></script>
  <script src="app.js"></script>          <!-- Link to the external JavaScript file -->
</head>
<body>
  <div ng-controller="SimpleController">
    <h1>{{ greeting }}</h1>
    <p>{{ message }}</p>
  </div>
</body>
</html>
```

2. External JavaScript File (app.js)

```
angular.module('myApp', [])
.controller('SimpleController', function($scope) {
  $scope.greeting = 'Hello, AngularJS!';
  $scope.message = 'This is a simple AngularJS example using an external JavaScript file.';
});
```


5.1.3 AngularJS's routine capability

5.1.3.1 \$routeProvider service from ngRoute

AngularJS's routing capability allows developers to create single-page applications (SPAs) where different views or components of the application are displayed based on the URL or user interactions without requiring a full page reload. This is achieved using the **ngRoute** module

1. Basic Routing with ngRoute

The ngRoute module is a built-in AngularJS module that provides routing and deep linking services. It allows you to map URL routes to specific views and controllers.

Setup:

1. Include AngularJS and ngRoute in your HTML:

```
<script src="angular.min.js"></script>
<script src="angular-route.min.js"></script>
```

2. Define the AngularJS module and configure routes: (app.js)

```
var app = angular.module('myApp', ['ngRoute']);

app.config(function($routeProvider) {
  $routeProvider
    .when('/home', {
      templateUrl: 'home.html',
      controller: 'HomeController'
    })
    .when('/about', {
      templateUrl: 'about.html',
      controller: 'AboutController'
    })
    .otherwise({
      redirectTo: '/home'
    });
});

app.controller('HomeController', function($scope) {
  $scope.message = 'Welcome to the Home Page!';
});

app.controller('AboutController', function($scope) {
  $scope.message = 'This is the About Page.';
});
```

3. Create the views:**home.html:**

```
<div>
  <h1>{{ message }}</h1>
</div>
```

about.html:

```
<div>
  <h1>{{ message }}</h1>
</div>
```

4. Set up the main HTML with a <ng-view> directive:**index.html:**

```
<html ng-app="myApp">
<head>
  <title>AngularJS Routing Example</title>
  <script src="angular.min.js"></script>
  <script src="angular-route.min.js"></script>
  <script src="app.js"></script>
</head>
<body>
  <div>
    <a href="#!/home">Home</a> |
    <a href="#!/about">About</a>
  </div>
  <div ng-view></div>
</body>
</html>
```

5.1.3.2 Navigating different pages

In AngularJS, navigating between different pages or views within a single-page application (SPA) is typically managed through routing. Routing allows you to change the view displayed to the user without requiring a full page reload.

Example:

```
<html>
<script src="angular.min.js"></script>
<script src="angular-route.js"></script>
<body ng-app="myApp">
<p><a href="#/!">Main</a></p>
<a href="#!red">Red</a>
<a href="#!green">Green</a>
<a href="#!blue">Blue</a>
<div ng-view></div>
<script>
var app = angular.module("myApp", ["ngRoute"]);
app.config(function($routeProvider) {
    $routeProvider
    .when("/", {
        templateUrl : "main.htm"
    })
    .when("/red", {
        templateUrl : "red.htm"
    })
    .when("/green", {
        templateUrl : "green.htm"
    })
    .when("/blue", {
        templateUrl : "blue.htm"
    });
});
</script>
```

<p>Click on the links to navigate to "red.htm", "green.htm", "blue.htm", or back to "main.htm"</p>

</body>

</html>

5.2 HTML DOM directives

The **HTML DOM** in AngularJS facilitates the directives that bind the application data with the attributes of HTML DOM elements. In this article, we will see such directives that help to bind the data to the HTML DOM element's attribute, along with their basic implementations through the illustrations.

5.2.1 ng-disabled, ng-show, ng-hide, ng-click

The following directives are used to bind application data to the attributes of HTML DOM elements –

Sr.No.	Name & Description
1	ng-disabled Disables a given control.
2	ng-show Shows a given control.
3	ng-hide Hides a given control.
4	ng-click Represents a AngularJS click event.

1. ng-disabled Directive:

- It disables the attribute of HTML elements. It binds the data to the disabled HTML element's attributes.

Example:

```
<html>
<head>
  <title>ng-disable Directive</title>
  <script src="angular.min.js">
  </script>
</head>
```

```

<body ng-app="">
  <div ng-init="IsChecked=true">
    <p>      <button ng-disabled="IsChecked">Click Me!</button>      </p>
    <p>      <input type="checkbox" ng-model="IsChecked"/>Button      </p>
    <p>      {{ IsChecked }}      </p>
  </div>
</body>
</html>

```

2. ng-show/ng-hide:

- Both the directives are used to **show** or **hide** the element of HTML. It depends upon either of the value of directives, i.e., either “*true*” or “*false*”.

Example:

```

<!DOCTYPE html>
<html>
<head>
  <script src= "angular.min.js"> </script>
</head>
<body style="text-align:center;">
  <div ng-app="">
    <h1 style="color:rgb(112, 0, 134)" ng-show="true"> Angular Js</h1>
    <p ng-show="false">ng-show is false</p>
    <p ng-hide="true">ng-hide is true</p>
    <p ng-hide="false"> Advance Web Designing      </p>
  </div>
</body>
</html>

```

3. **ng-click**

The ng-click Directive in AngularJS is used to apply custom behavior when an element is clicked. It can be used to show/hide some element or it can pop up an alert when the button is clicked.

Syntax:

```
<element ng-click="expression"> Contents... </element>
```

Parameter Value:

- expression:** It specifies when the particular element is clicked then the specific expression will be evaluated.

Supported tag: It is supported by all the elements in HTML.

Example:

```
<html lang="en">
  <head>
    <title>Events</title>
    <script src="angular.min.js"></script>
    <style>
      .num {
        color:rgb(104, 0, 165);
        padding:10px;
        font-family: Arial, Helvetica, sans-serif;
        font-size: 50px;
      }
    </style>
  </head>
  <body ng-app="mymod">
    <div ng-controller="mycontrol">
      <input type="button" value="Like" ng-click="Number = Number + 1 ">
      <input type="button" value="DisLike" ng-click="Number = Number - 1 ">
      <div class="num">{{Number}}</div>
    </div>
    <script>
      var app = angular.module('mymod', []);
      app.controller('mycontrol', function($scope) {
        $scope.Number=0;
      });
    </script>
  </body>
</html>
```

AngularJS Events:

- **Events** in AngularJS can be used to perform particular tasks, based on the action taken. Both Angular Event & the HTML Event will be executed & will not overwrite with an HTML Event.
- It can be added using the Directives mentioned below:

ng-mousemove	The movement of the mouse leads to the execution of the event.
ng-mouseup	Movement of the mouse upwards leads to the execution of the event.
ng-mousedown	Movement of the mouse downwards leads to the execution of the event.
ng-mouseenter	Click of the mouse button leads to the execution of the event.
ng-mouseover	Hovering the mouse leads to the execution of the event.
ng-cut	Cut operation leads to the execution of the event.
ng-copy	Copy operation leads to the execution of the event.
ng-keypress	Press of key leads to the execution of the event.
ng-keyup	Press of upward arrow key leads to the execution of the event.
ng-keydown	Press of downward arrow key leads to the execution of the event.
ng-click	Single click leads to the execution of the event.
ng-dblclick	Double click leads to the execution of the event.
ng-blur	Fired when an HTML element loses its focus.
ng-change	It is used whenever the value of an input element changes.
ng-focus	It is used to apply custom behavior when an element is focused.
ng-paste	It is used to specify custom behavior functions when the text in input fields is pasted into an HTML element.
ng-mouseleave	It is used to apply custom behavior when a mouse-leave event occurs on a specific HTML element.

5.2.3 Forms (Events, Data validation, ng-click)

AngularJS Forms:

Forms are collection of controls that is input field, buttons, checkbox and these can be validated real time. As soon as a user of the form completes writing a field and moves to the next one it gets validated and suggests the user where he might have went wrong.

So a form can be consisting of the many number of controls however we will be focusing on

1. Input field
2. Check box
3. Radio box
4. Button
5. Select Box(Dropdowns)

Example:

```

<html ng-app="studentApp">
<head>
  <script src="angular.js"></script>
</head>
<body ng-controller="studentController">
  <h1>Student Information:</h1>
  <form ng-submit="submitStudnetForm()" >
    <table>
      <tr>
        <td> <label for="firstName" >First Name: </label>    </td>
        <td> <input type="text" id="firstName" ng-model="student.firstName" />    </td>
      </tr>
      <tr>
        <td> <label for="lastName">Last Name</label>          </td>
        <td> <input type="text" id="lastName" ng-model="student.lastName" />    </td>
      </tr>
      <tr>
        <td> <label for="dob" >DoB</label>          </td>
        <td> <input type="date" id="dob" ng-model="student.DoB" />    </td>
      </tr>
      <tr>
        <td> <label for="gender" >Gender</label> </td>
        <td>
          <select id="gender" ng-model="student.gender">
            <option value="male">Male</option>
            <option value="female">Female</option>
          </select>
        </td>
      </tr>
      <tr>
        <td><span>Training Type:</span></td>
        <td><label><input value="online" type="radio" name="training" ng-model="
"student.trainingType"/> Online</label>
          <label><input value="onsite" type="radio" name="training" ng-model =
"student.trainingType" />OnSite</label>
        </td>
      </tr>
      <tr>
        <td>
          <span>Subjects</span><br />
        </td>
        <td>
          <label><input type="checkbox" ng-model="student.maths" />Maths</label>
          <label><input type="checkbox" ng-model="student.physics" />Physics</label>
        </td>
      </tr>
    </table>
  </form>
</body>
</html>

```



```
        <label><input type="checkbox" ng-model="student.chemistry" />Chemistry</label>
    </td>
</tr>
<tr>
    <td> <input type="reset" ng-click="resetForm()" value="Reset" /> </td>
    <td> <input type="submit" value="Submit" /> </td>
</tr>
</table>
</form>
<script>
```

```
//1. create app module
```

```
var studentApp = angular.module('studentApp', []);
```

```
//2. create controller
```

```
studentApp.controller("studentController", function ($scope, $http) {
```

```
    //3. attach originalStudent model object
```

```
    $scope.originalStudent = {
        firstName: 'pooja',
        lastName: 'soni',
        DoB: new Date('01/31/1980'),
        gender: 'Female',
        trainingType: 'online',
        maths: false,
        physics: true,
        chemistry: true
    };

```

```
//4. copy originalStudent to student. student will be bind to a form
```

```
$scope.student = angular.copy($scope.originalStudent);
```

```
//5. create submitStudentForm() function. This will be called when user submits the form
```

```
$scope.submitStudentForm = function () {
    var onSuccess = function (data, status, headers, config) {
        alert('Student saved successfully.');
```

```
        $http.post('/student/submitData', { student:$scope.student })
            .success(onSuccess)
            .error(onError);
    };

    //6. create resetForm() function. This will be called on Reset button click.

    $scope.resetForm = function () {
        $scope.student = angular.copy($scope.OriginalStudent);
    };
});
</script>
</body>
</html>
```

Validate Data

The following can be used to track error.

- **\$dirty** – states that value has been changed.
- **\$invalid** – states that value entered is invalid.
- **\$error** – states the exact error.

Example

```
<html lang="en">
<head>
    <title>Validation</title>
    <link rel="stylesheet" href="bootstrap.min.css">
    <script src="angular.min.js"></script>
</head>
<style>
    .red-text{
        color:red;
    }
</style>
<body>
<div class="container" ng-app="myApp">
    <div class="col-md-5">
        <div class="form-area" ng-controller="formCtrl">
            <form role="form" name="ReisterForm" novalidate>
```

```

<br style="clear:both">
<h3 style="margin-bottom: 25px; text-align: center;">Reisteration Form</h3>
<div class="form-group"> <input name="name" ng-model="name" type="text"
    ng-required="true" autocomplete="off" placeholder="Name" class="form-control">
    <span      class="red-text"      ng-if="ReisterForm.name.$error.required      &&
    ReisterForm.name.$dirty">Name is a required </span>
</div>
<div class="form-group">
<textarea  name="address"  ng-model="address"  type="text"  ng-required="true"  auto
complete="off" placeholder="Address" class="form-control"></textarea>
<span      class="red-text"      ng-if="ReisterForm.address.$error.required      &&
ReisterForm.address.$dirty"> Address is a required </span>

</div>
<div class="form-group">
    <input name="email" ng-model="email" id="eml" type="text" ng-pattern="eml_add" ng-
required="true" autocomplete="off" placeholder="Email" class="form-control">

        <span  class="red-text"  ng-if="ReisterForm.email.$error.required  &&
ReisterForm.email.$dirty">Email is a required field</span>

        <span class="red-text" ng-show="ReisterForm.email.$error.pattern">Invalid Email</span>
</div>
<div class="form-group">
    <input name="contact" ng-model="contact" id="c_num" type="text" ng-required="true" ng-
pattern="ph_numbr" autocomplete="off" placeholder="Contact Number" class="form-control">
    <span      class="red-text"      ng-if="ReisterForm.contact.$error.required      &&
ReisterForm.contact.$dirty">Contact number is a required field</span>
    <span class="red-text" ng-show="ReisterForm.contact.$error.pattern">Please enter a 10 digit
number</span>
</div>
    <button type="button" id="submit" name="submit" class="btn btn-primary pull-right" ng-
disabled="ReisterForm.$invalid">Submit Form</button>

```

```
</form>
</div>
</div>
</div>
<script>
  var app = angular.module("myApp", []);
  app.controller("formCtrl", function($scope) {
    $scope.ph_numbr = /^\\+?\\d{10}$/;
    $scope.eml_add = /^[^\\s@]+@[^\\s@]+\\.\\.[^\\s@]{2,}$/;
  });
</script>
</body>
</html>
```

AngularJS - Includes

HTML does not support embedding HTML pages within the HTML page. To achieve this functionality, we can use one of the following options –

Using Ajax – Make a server call to get the corresponding HTML page and set it in the innerHTML of HTML control.

Using Server Side Includes – JSP, PHP and other web side server technologies can include HTML pages within a dynamic page.

Using AngularJS, we can embed HTML pages within an HTML page using ng-include directive.

AngularJS Routing

We can build Single Page Application (SPA) with AngularJS. It is a web app that loads a single HTML page and dynamically updates that page as the user interacts with the web app.

AngularJS supports SPA using routing module ngRoute. This routing module acts based on the url. When a user requests a specific url, the routing engine captures that url and renders the view based on the defined routing rules.

Let's see how to implement simple routing in AngularJS application.

```
<html>

  <script src=" angular.min.js"></script>

  <script src=" angular-route.js"></script>
```

```
<body ng-app="myApp">

  <p><a href="#/!">Main</a></p>

  <a href="#!red">Red</a>

  <a href="#!green">Green</a>

  <a href="#!blue">Blue</a>

  <div ng-view></div>

  <script>

var app = angular.module("myApp", ["ngRoute"]);

app.config(function($routeProvider) {

  $routeProvider

    .when("/", {

      templateUrl : "main.htm"

    })

    .when("/red", {

      templateUrl : "red.htm"

    })

    .when("/green", {

      templateUrl : "green.htm"

    })

    .when("/blue", {

      templateUrl : "blue.htm"

    });

});

</script>
```

<p>Click on the links to navigate to "red.htm", "green.htm", "blue.htm", or back to "main.htm"</p>

</body>

</html>

Example of a Basic AngularJS SPA

1. Create the AngularJS Application: (index.html)

```
<html ng-app="myApp">
<head>
  <title>AngularJS SPA</title>
  <script src="angular.min.js"></script>
  <script src="angular-route.min.js"></script>
  <script src="app.js"></script>
</head>
<body>
  <div ng-view></div>
</body>
</html>
```

2. Define the Application and Routing: (app.js)

```
var app = angular.module('myApp', ['ngRoute']);
app.config(function($routeProvider) {
  $routeProvider
    .when('/home', {
      templateUrl: 'home.html',
      controller: 'HomeCtrl'
    })
    .when('/about', {
      templateUrl: 'about.html',
      controller: 'AboutCtrl'
    })
    .otherwise({
```

```
        redirectTo: '/home'
    });
});
app.controller('HomeCtrl', function($scope) {
    $scope.message = 'Welcome to the Home Page!';
});
app.controller('AboutCtrl', function($scope) {
    $scope.info = 'This is the About Page.';
});
```

3. Create Views:

home.html:

```
<div>

    <h1>{{ message }}</h1>

    <p><a href="#!/about">Go to About Page</a></p>

</div>
```

about.html:

```
<div>

    <h1>{{ info }}</h1>

    <p><a href="#!/home">Go to Home Page</a></p>

</div>
```